

RAG Document Reader

System Architecture and Implementation Documentation

Field	Details
Status	Active development
Version	Work in progress - April 2026
Document type	Internal technical documentation
Primary stack	FastAPI backend, Next.js 14 frontend, LangGraph pipeline

1. Objective

The RAG Document Reader is a lightweight full-stack application for uploading PDF or DOCX files, indexing their extracted content, and answering user questions against the uploaded document. The system combines deterministic document ingestion, chunking, hybrid retrieval, and a grounded LLM response pipeline so users can ask natural-language questions while still seeing the source chunks used to generate each answer.

2. Current Scope

- Upload and process PDF and DOCX files through the FastAPI backend.
- Extract selectable PDF text with PyPDFLoader and fall back to OCR when needed.
- Read DOCX files through LangChain's Docx2txtLoader.
- Split documents into overlapping chunks for retrieval-friendly context windows.
- Combine dense FAISS vector search with sparse BM25 keyword retrieval.
- Use a LangGraph StateGraph to run retrieval and answer generation as a structured pipeline.
- Display answers, source chunks, document status, and token usage in a Next.js interface.

3. High-Level Architecture

Layer	Implementation	Responsibility
Frontend	Next.js 14, TypeScript, React	Upload documents, submit questions, show results and retrieved chunks.
API	FastAPI	Expose health, document upload, ask, and status endpoints.
Document loaders	PyPDFLoader, Docx2txtLoader, pdf2image, pytesseract	Extract text from PDFs and DOCX files, including OCR fallback for scanned PDFs.
Chunking	RecursiveCharacterTextSplitter	Create 1800-character chunks with 350-character overlap.
Retrieval	FAISS index plus BM25	Merge semantic and lexical matches with reciprocal-rank fusion.
Orchestration	LangGraph StateGraph	Run retrieve and generate nodes in a deterministic sequence.
LLM	gpt-4o-mini via ChatOpenAI	Generate grounded answers from retrieved context at temperature 0.2.

4. Document Ingestion and OCR

The backend inspects the uploaded file extension and accepts only PDF and DOCX files. DOCX content is read directly with Docx2txtLoader. PDF files are first parsed with PyPDFLoader; if the extracted text is too sparse, the pipeline converts pages to images and applies pytesseract OCR. This keeps the app useful for both digital documents and scanned legacy files.

- Supported upload types: .pdf and .docx.
- PDF fallback threshold: extracted text must contain more than 500 non-whitespace characters.

- OCR dependencies: pdf2image, pytesseract, Poppler, and a configured Tesseract executable.
- Temporary upload files are deleted after processing.

5. Chunking Strategy

Text is split with RecursiveCharacterTextSplitter using a hierarchy of separators: blank lines, line breaks, sentence boundaries, semicolons, spaces, and finally characters. This favors semantically useful chunks while still guaranteeing that long text can be indexed.

Parameter	Value	Purpose
chunk_size	1800 characters	Captures enough context for definitions, clauses, tables, or explanations.
chunk_overlap	350 characters	Preserves continuity when a sentence or concept crosses chunk boundaries.
separators	\n\n, \n, . , ; , space, empty string	Falls back from larger logical units to smaller units as needed.

6. Hybrid Retrieval

The active backend builds a FAISS IndexFlatIP index from OpenAI embedding vectors and normalizes vectors for cosine-style inner-product search. In parallel, BM25 provides lexical matching for exact terms, regulation numbers, acronyms, and other keyword-heavy queries. Results are merged through reciprocal-rank fusion with a 0.6 semantic weight and a 0.4 BM25 weight.

Retriever	Current behavior	Why it matters
FAISS dense search	Embeds chunks with text-embedding-ada-002 and searches normalized vectors.	Finds conceptually similar passages even when wording differs.
BM25 sparse search	Lowercases and splits text, then returns the top keyword matches.	Handles exact terms, section labels, identifiers, and named entities.
Fusion	Combines ranked results using weighted reciprocal-rank scoring.	Balances semantic relevance with precise keyword evidence.

7. Retrieval-Generation Graph

The query flow is implemented as a LangGraph StateGraph. It is a stateful execution pipeline rather than a domain knowledge graph. The state contains the user question, retrieved context blocks, and final answer.

1. Retrieve node expands the user query to prioritize definitions, clauses, tables, rules, and exact phrases.
2. Hybrid retrieval returns the most relevant chunks with source, page, content, and relevance metadata.
3. Generate node assembles the retrieved context and asks gpt-4o-mini to answer only from that context.
4. The graph runs sequentially as Retrieve -> Generate -> End.

8. API Surface

Endpoint	Method	Purpose
/health	GET	Basic liveness check returning { ok: true }.
/	GET	Root metadata with endpoint list and API status.
/status	GET	Current document name, chunk count, token usage, and retrieval method.
/documents	POST	Upload and index a PDF or DOCX document.
/ask	POST	Ask a question against the currently indexed document.

9. Frontend Experience

The frontend is a Next.js 14 application that defaults to `http://localhost:8080` for API calls unless `NEXT_PUBLIC_API_BASE_URL` is provided. Users select a PDF or DOCX file, process it, ask a question, and review the answer together with retrieved source chunks. The interface also shows cumulative input and output token usage returned by the backend.

10. Runtime Configuration

Setting	Used by	Notes
<code>OPENAI_API_KEY</code>	Backend	Required for embeddings and chat generation.
<code>POPPLER_PATH</code>	Backend OCR	Optional path used by pdf2image when Poppler is not on PATH.
<code>TESSERACT_CMD</code>	Backend OCR	Optional path to the Tesseract executable; defaults to the common Windows install path.
<code>NEXT_PUBLIC_API_BASE_URL</code>	Frontend	Optional backend URL override for the Next.js app.

11. Local Execution

Backend:

```
.\venv\Scripts\activate
pip install -r requirements.txt
uvicorn backend.main:app --reload --port 8080
```

Frontend:

```
cd frontend
npm install
npm run dev
```

12. Known Implementation Notes

- The app currently stores the indexed document in process memory, so a backend restart clears the active index.
- Only one uploaded document is active at a time in the current runtime object.
- OCR requires local Poppler and Tesseract availability in addition to Python packages.
- The backend tracks LLM input/output token usage for question answering; embedding token accounting is not exposed by the active `main.py` response.
- Future enhancements may include persistent vector storage, multi-document workspaces, reranking, streaming answers, and authentication.